

FSD WORKSHOP:

CODE: 25VA351

Experiment 1:

Create a Custom EventEmitter that Triggers "greet" and "exit"

Aim: To create a custom EventEmitter in Node.js and trigger custom events named **greet** and **exit**.

Theory: Node.js provides the **events** module that allows objects to emit named events. The **EventEmitter** class is used to create and handle custom events. An event listener executes automatically whenever the corresponding event is emitted.

Node.js is an event-driven runtime environment. Many operations, such as file reading, HTTP requests, timers, and user-defined actions, are performed using **events**.

An event is an action or occurrence that happens during the execution of a program. For example:

- A user clicks a button.
- A file is opened.
- A server receives a request.
- A timer completes.
- A custom event such as **greet** or **exit** is triggered.

Node.js provides the built-in **events** module to create and manage events.

The **EventEmitter** class is the core class of this module. It allows developers to:

- Create custom events.
- Register event listeners.
- Trigger (emit) events.
- Pass data to event listeners.

When an event is emitted, all the registered listeners for that event execute automatically.

Algorithm

1. Import the **events** module.
2. Create an object of the **EventEmitter** class.
3. Register listeners for the **greet** and **exit** events.
4. Emit the events.
5. Display the corresponding messages.

CODE:

1. %%writefile EventEmitter.js: java script file
2. `const EventEmitter = require('events');`: This imports the **events module**. (type of library)
3. `const myEmitter = new EventEmitter();` create object
4. `myEmitter.on('greet', (name) => { : [ on() means "Start listening.", even name is greet]`
5. `console.log(`Hello, ${name}! Welcome to Node.js.`);`: Prints the greeting.
6. `myEmitter.on('exit', () => { : [ another event , exit]`
7. `console.log("Application Closed.");` closed
8. `myEmitter.emit('greet', 'Tarun');` exit mean even run

Create a Custom EventEmitter that Triggers "greet" and "exit"

[2]

✓ 0s



%%writefile EventEmitter.js

```
const EventEmitter = require('events');

const myEmitter = new EventEmitter();

myEmitter.on('greet', (name) => {
  console.log(`Hello, ${name}! Welcome to Node.js.`);
});

myEmitter.on('exit', () => {
  console.log("Application Closed.");
});

myEmitter.emit('greet', 'Tarun');
myEmitter.emit('exit');
```

✓

... Writing EventEmitter.js

[3]

✓ 0s

!node EventEmitter.js

✓

Hello, Tarun! Welcome to Node.js.
Application Closed.

Simulate DOM-like Event Handling in Node.js Using Events

Aim: To simulate browser-like DOM event handling in Node.js using the EventEmitter class.

Theory: Browsers use events such as click, mouseover, and keydown to interact with users. Since Node.js has no DOM, similar behavior can be simulated using custom events with EventEmitter.

Algorithm

1. Import the events module.
2. Create an EventEmitter object.
3. Register listeners for click and mouseover events.
4. Emit the events.
5. Observe the execution of event handlers.

Note:

Node.js does not have a browser or a DOM, so events like click and mouseover do not exist naturally. To simulate this behavior, Node.js uses the **EventEmitter** class from the **events** module. We create custom events named "click" and "mouseover" and trigger them manually using emit()

Example:

Imagine a classroom.

- Teacher says "Stand Up" → Students stand up.
- Teacher says "Sit Down" → Students sit down.

Here:

- Teacher = EventEmitter
- Command = Event
- Students = Event Listener

Note:

!node eventEmitter.js	eventEmitter.js	Demonstrates a custom EventEmitter with greet and exit events.	Hello, Tarun! Welcome to Node.js. Application Closed.
!node domSimulation.js	domSimulation.js	Simulates DOM events (click, mouseover) using EventEmitter.	Button Clicked! Mouse is over the button.

Code:

```
1] 0s ▶ %%writefile domSimulation.js

// Import EventEmitter class
const EventEmitter = require('events');

// Create Button class
class Button extends EventEmitter {}

// Create object
const button = new Button();

// Click Event Listener
button.on('click', () => {
  console.log("Button Clicked!");
});

// Mouseover Event Listener
button.on('mouseover', () => {
  console.log("Mouse is over the button.");
});

// Trigger Events
button.emit('click');
button.emit('mouseover');

... Writing domSimulation.js

1] 0s !node domSimulation.js

Button Clicked!
Mouse is over the button.
```

Visualize the Event Loop Using setTimeout(), setImmediate(), and process.nextTick()

Aim: To understand the execution order of `process.nextTick()`, `setTimeout()`, and `setImmediate()` in the Node.js event loop.

Theory: Node.js executes asynchronous operations through its **Event Loop**.

- **process.nextTick()** executes before the event loop continues.
- **setTimeout()** executes after the specified delay (Timers phase).

- **setImmediate()** executes during the Check phase of the event loop.

Flow Diagram:

Synchronous Code



process.nextTick()



setTimeout()



setImmediate()